

Packet Flow and Architecture

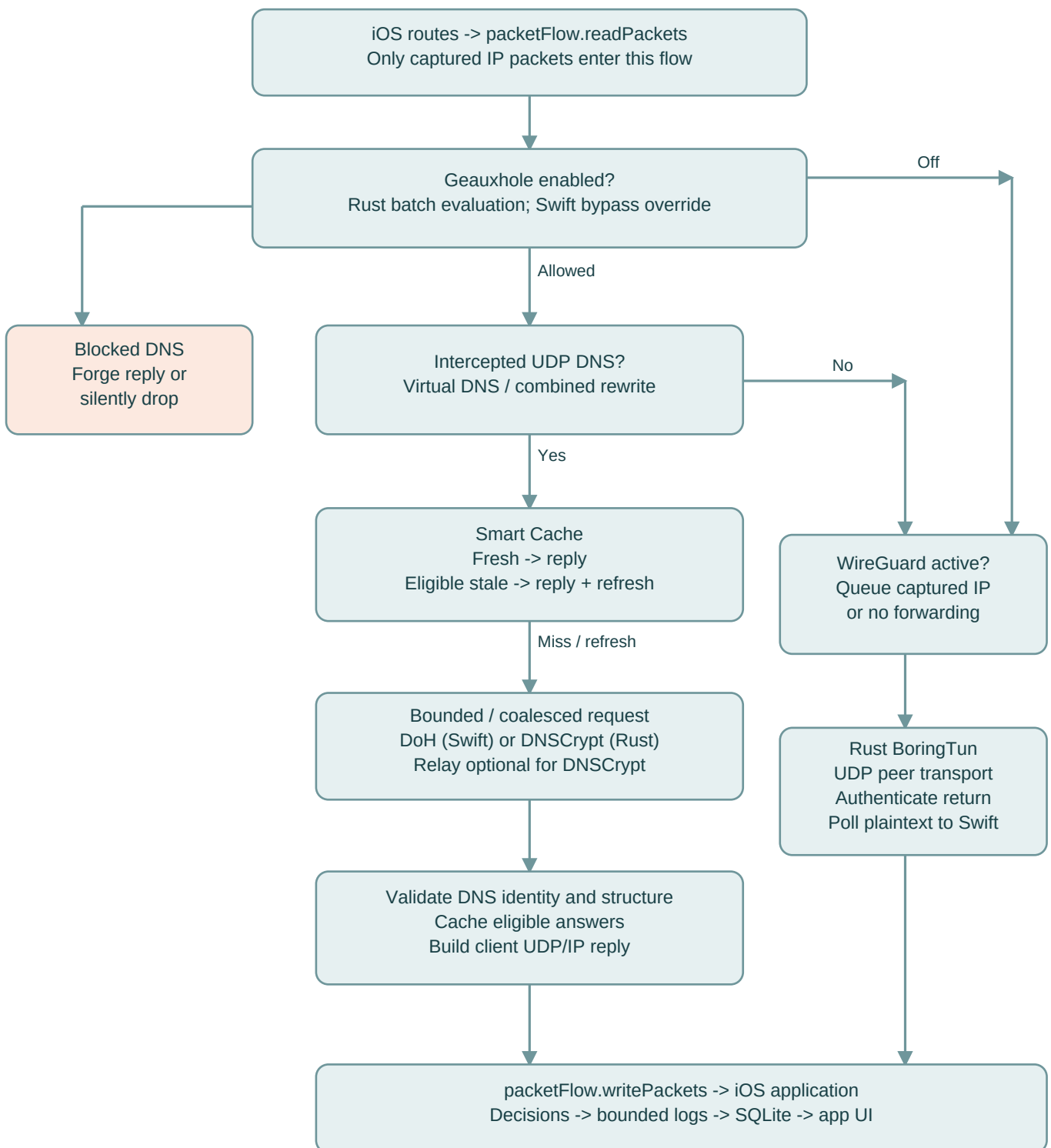
Practical reference / September 2026

- 01 Architecture and evidence**
- 02 Packet flow and routing decisions**
- 03 Rule compilation and matching algorithm**
- 04 Swift / C / Rust memory contract**
- 05 DNS processing, validation, and replies**
- 06 Cache and concurrency algorithms**
- 07 DoH and DNSCrypt transports**
- 08 WireGuard processing and pacing**
- 09 Persistence, reporting, and lifecycle**
- 10 Implementation limits and maintenance checklist**

Based on the app and website archives supplied for this documentation update. Source descriptions include implementation limits and should be reviewed when code changes.

Packet flow at a glance

Routing, local DNS, and WireGuard are separate branches. See the following chapters for policy and unsupported-transport details.



Architecture and evidence

Engineering reference for geaux-main(4)(5).zip, inspected 14 September 2026. Paths are relative to the archive's geaux-main root. Descriptions distinguish source behavior from inferred limitations; no Swift or Rust implementation files were changed.

Layer	Responsibilities
SwiftUI app	Edits configuration, controls VPN state, displays bounded traffic history and aggregates, exports diagnostics.
Shared GeauxCore	Configuration persistence/compilation orchestration, DNS validation and transports, cache utilities, database, metrics, secrets.
PacketTunnelProvider	Owens NetworkExtension packet flow, routing settings, batch evaluation, DNS dispatch and replies, engine generation and recovery.
C ABI / geaux_rust.h	Packet arrays and evaluation structs, opaque WireGuard and cancellation handles, DNSCrypt completion callback.
Rust lib.rs	Rule compilation and memory mapping; UDP DNS packet inspection; block response forging; DNSCrypt crypto/transport.
Rust wireguard.rs	BoringTun sessions, UDP socket workers, routing policy, queues, pacing, return-packet channels.

Control plane

UI changes pass through ConfigurationManager/ConfigurationWorker into shared persisted configuration and compiled rule generations. The app asks the extension to reload; candidate configuration and rules must be accepted before publication. VPNManager coordinates the selected mode and iOS tunnel lifecycle.

Data plane

iOS delivers captured IP packet batches. Swift optionally asks Rust to classify DNS, chooses local reply/upstream DNS/WireGuard/drop, and returns packets through packetFlow.writePackets. Resolver results are validated before caching and delivery. Logging is asynchronous and does not store packet payloads as a packet capture.

Reference convention

Each page names the source files and key symbols supporting it. The supplied-source manifest in the complete package records SHA-256 hashes so this documentation can be compared with later code. Statements about defaults refer to defaultConfiguration(), not migrated user settings.

Source: apple/GeauxCore/ConfigurationManager.swift; ConfigurationWorker.swift; VPNManager.swift; apple/GeauxExtension/PacketTunnelProvider.swift; geaux-rust/geaux_rust.h

Packet flow and routing decisions

The OS route table decides which packets reach the extension. The DNS filter cannot inspect a packet that never enters packetFlow.

Stage	Branch / result
1. Route selection	DNS-only includes 240.0.0.2/32 and fd00::2/128 and installs them as DNS servers. WireGuard modes derive routes from AllowedIPs; combined mode additionally includes virtual DNS routes.
2. Batch read	readPackets captures one FastPathConfig snapshot. Geauxhole-enabled modes call geaux_evaluate_packet_batch once for the packet batch.
3. Rule decision	A Swift filtering-bypass match overrides the returned block action. Otherwise Reject/Reject No Drop append forged replies; Reject Drop stops the packet.
4. DNS dispatch	Allowed intercepted UDP DNS to a virtual DNS address, or captured hardcoded DNS in combined rewrite mode, enters cache/resolver handling.
5. Other captured packets	If WireGuard is active and the packet was not consumed, append it to the outbound batch. DNS-only has no general IP forwarding implementation.
6. Return path	Local DNS replies use the original source/destination addresses and ports in reverse. WireGuard return packets are authenticated/decrypted in Rust and polled into Swift for injection.

Route exclusions and resolver egress

applyNetworkSettings excludes the WireGuard endpoint. For DNSCrypt it extracts and excludes the relay IP, or the resolver IP when no relay is configured. DoH uses an extension-owned URLSession. There is no explicit call that encapsulates encrypted resolver requests through geaux_wg_queue_encrypt; do not describe combined mode as guaranteeing DNS-over-WireGuard.

IP and DNS boundaries

UDP parsing validates lengths and rejects unsupported fragmented paths. processPacket drops DNS-over-TCP or fragmented DNS when interception policy applies. Non-DNS TCP can still be carried by WireGuard as IP traffic. App-owned encrypted DNS is not decrypted or inspected as DNS.

WireGuard-only rewrite gap: source inference

readPackets calls processPacket with eval:nil in wireguardOnly mode. processPacket initializes isDns=false and parsed4/parsed6=nil, then populates them only from a valid evaluation. Consequently its if isDns rewrite block is not reached in that mode. The configured feature is intended to rewrite to profile DNS, but this snapshot does not establish that behavior. This is a documented implementation gap, not a tested fix.

Source: apple/GeauxExtension/PacketTunnelProvider.swift: applyNetworkSettings, readPackets, processPacket, batchQueueEncrypt; apple/GeauxCore/IPPacketParser.swift

Rule compilation and matching algorithm

Rules are compiled into compact sorted integer records. This is a domain matcher, not a URL filter or full Adblock parser.

Normalization

Rust `process_line_bytes` trims whitespace and skips comments and unsupported entries. Plain domain lines are exact. `*.domain` and `||domain^` are suffix rules. Common hosts lines strip the leading IP and select the next domain as a suffix rule. URLs, IP literals, Adblock exceptions and modifiers are rejected or skipped. UI custom rules use `DomainRuleCanonicalizer`; ASCII domain forms are the safest shared representation.

Packed record layout

Each 64-bit record stores a 55-bit lowercase FNV-1a hash in bits 0-54, an exact-match flag in bit 55, a two-bit action in bits 56-57, and a six-bit list ID in bits 58-63. Hash input is the normalized domain. This hash is a compact lookup key, not a cryptographic digest; collisions are theoretically possible and names are not rechecked in the Rust hash lookup.

Sort, conflicts, and publication

Compilation sorts by the lower 56 bits, then higher list ID, then action priority `Drop > Reject > Reject No Drop > Allow`. Duplicate lookup keys retain the first record. Custom rules use list ID 63; entropy uses 62 as a reported match ID. Compiled rule data is activated through `geaux_init_rules` and memory mapped for reads.

Lookup order

For an ordinary question, the evaluator hashes the complete name, checks the exact-key variant, then the suffix variant. It walks successively shorter label suffixes and binary-searches each. The first match wins. A more-specific rule can therefore win before a less-specific custom rule; custom priority resolves identical lookup keys, not every possible overlap.

Cost model

Sorting is $O(N \log N)$ for N compiled records. Each lookup uses $O(\log N)$ binary searches per candidate suffix, plus bytes hashed. Each suffix is hashed again, so total name hashing is the sum of suffix lengths. Mapped storage uses eight bytes per compiled record, plus mapping/process overhead.

Shannon entropy

If no list matched and the setting is enabled, calculate $H = -\sum(p_i * \log_2(p_i))$ over the extracted domain bytes. $H > 4.5$ sets `Reject` and match ID 62. This is a character-distribution heuristic; it does not prove malware. Explicit `Allow` matches skip the heuristic, and Swift filtering bypass can override a returned block decision.

Source: `geaux-rust/src/lib.rs: process_line_bytes, fnv1a_hash_55, geaux_compile_rules, geaux_init_rules, geaux_evaluate_packet_batch, calculate_shannon_entropy`; `apple/GeauxCore/RuleGeneration.swift`

Swift / C / Rust memory contract

The C header is the ABI contract. Buffer lifetimes and ownership differ between synchronous evaluation and asynchronous transport work.

Interface	Ownership and result
<code>geaux_evaluate_packet_batch</code>	Swift retains NSData owners for each input pointer through the synchronous call. Rust fills the caller-owned GeauxEvaluation array. Swift consumes the results before releasing owners.
<code>GeauxEvaluation</code>	Carries domain buffer[256], action/list ID, transport metadata, offsets, qtype, and forged buffer[512] with explicit lengths. Consumers must respect lengths and validity flags.
<code>geaux_wg_queue_encrypt_batch</code>	Swift borrows stable pointers during the call. Rust copies accepted bytes into budgeted queue ownership before return. Swift can release its owners afterward.
<code>geaux_wg_poll_tun_batch_blocking</code>	Swift supplies fixed-stride output storage and lengths. Rust waits for the first packet or shutdown, then drains available packets up to the limit. Swift copies valid slots into Data.
<code>geaux_resolve_dnscrypt</code>	Swift passes a retained continuation context and cancellation token. Rust owns asynchronous work; callback bytes are copied to Swift Data during callback.
<code>geaux_get_dnscrypt_ip</code>	Returns an allocated C string. Swift copies the string then calls <code>geaux_free_string</code> .

WireGuard handle shutdown

WireGuardSocketManager serializes outbound/control borrows with `ffiLock`. Its independent poller owns a borrow until it exits. `stopAndWait` prevents new work, calls `geaux_wg_shutdown`, and waits for the poller. Only after borrows drain may the provider call `geaux_wg_free`. A timeout must not be treated as permission to free live state.

DNSEncrypt exactly-once completion

ContinuationWrapper uses a lock to allow one resume from success, timeout, or cancellation. The callback consumes the retained context even if a timeout already resumed the continuation. The cancellation token remains alive with the wrapper until asynchronous callback ownership is released.

Batching bounds

Evaluation reuses scratch arrays and drops retained payload owners after each batch; unusually large scratch buffers are released above 1,024 entries. The WireGuard receive poller allocates 256 slots of 4,096 bytes and splits returned packets into IPv4 and IPv6 write batches.

Source: `geaux-rust/geaux_rust.h`; `geaux-rust/src/lib.rs`; `geaux-rust/src/wireguard.rs`: `geaux_wg_queue_encrypt_batch`, `geaux_wg_poll_tun_batch_blocking`; `apple/GeauxCore/WireGuardSocketManager.swift`; `DNSEncryptResolver.swift`

DNS processing, validation, and replies

The intercepted path preserves DNS type and request semantics while enforcing bounded work and validating upstream responses.

Before resolution

The provider extracts qname, qtype, UDP endpoints, and question offsets. A or AAAA is not silently converted to another query type. Intercepted local-discovery names are refused early when ignoreLocalDiscovery is enabled. Metadata tracks EDNS payload size and DNSSEC-related request flags.

Question and response validation

DNSMessageValidator requires exactly one IN-class question. A response must match transaction ID, opcode, canonical name, type, and class. It rejects malformed structures and truncated upstream DNS messages. Compression decoding has bounds and loop checks. Unknown resource-record payloads are opaque after structural bounds checks.

Outcome classification

RCODE 0 with answers is positive; RCODE 0 without answers and NXDOMAIN are negative. Other extended/ordinary RCODEs become typed failures. Negative cache lifetime uses the authority SOA minimum and record TTL bounds; missing usable negative lifetime yields no cache storage. Structural validation is not local DNSSEC signature verification.

Client response

After validation or a cache hit, record TTLs are adjusted, the client's transaction ID is restored, and UDP/IP headers are rebuilt with reversed endpoints. responseForClient limits payload to the client allowance and at most 1,232 bytes. Oversize replies are truncated, which can require a client TCP retry that the intercepted path does not implement.

Block and failure response distinction

Rust Reject and Reject No Drop both forge an NXDOMAIN-style reply with a 60-second SOA when the result fits the 512-byte forged buffer. Swift emits neither for Reject Drop. Resolver errors instead generate a SERVFAIL response with a separate DNSFailureReason. Geauxhole's Swift helper constructs other local replies; its zero-TTL SOA is not the Rust block TTL.

Malformed and unsupported data

Invalid/unrecognized UDP metadata cannot become a normal validated upstream DNS request. Non-IN questions, response mismatches, malformed records, upstream failures, and unsupported required TCP handling are not successful answers and must not be described as cache successes.

Source: apple/GeauxCore/DNSMessageValidator.swift; DNSQueryMetadata.swift; DNSRecordTTL.swift; DNSFailureReason.swift; Geauxhole.swift; apple/GeauxExtension/PacketTunnelProvider.swift: processPacket, responseForClient, makeServerFailureResponse; geaux-rust/src/lib.rs

Cache and concurrency algorithms

The Smart Cache is in extension memory. The SQLite traffic database is a separate store and does not supply DNS answers.

Cache key isolation

CacheKey includes lowercase domain, qtype, IN qclass, active resolver fingerprint, request flags, and the complete DNS request bytes except the first two transaction-ID bytes. EDNS options and case differences in the request wire image therefore prevent unsafe sharing. Resolver fingerprints include ID, protocol, endpoint/stamp, relay, and relay bypass domains.

Fresh lifetime and eviction

Fresh lifetime is $\min(\text{validated response lifetime}, \text{customTTL})$. A zero cap bypasses cache read/write.

ThreadSafeLRUCache combines a dictionary and doubly linked list under a lock: get/set move entries to the head; eviction removes the tail. It is bounded to 1,000 entries and an estimated 4 MiB cost, including key and record metadata. The byte estimate is not a measured process RSS limit.

Stale while refreshing

CachedResponse uses ContinuousClock for elapsed time and expiry. Eligible positive, non-AD-marked responses remain stale-eligible for 900 seconds after expiry. A stale hit is returned immediately with 30-second record TTLs while a bounded refresh is launched. Negative or AD-marked responses cannot take that stale path. Fresh hits age record TTLs. Incremental sweeping bounds inspections instead of scanning the full cache.

Coalescing and overload

DNSRequestCoordinator is an actor keyed like the cache. Concurrent identical requests share one Task; each client still receives its own transaction ID. Limits are 128 distinct upstream requests, 256 total waiters, 32 per key, 2 MiB estimated waiter bytes, and 512 KiB per key. A pre-Task ingress limiter bounds admitted packet work too.

Failure backoff and generations

Failed keys are suppressed for two seconds, with at most 512 failure entries. This is separate from DNS negative caching. Advancing the generation cancels older coordinator work; completion generation checks prevent old answers entering the current path after reload or stop.

Clearing behavior

CLEAR_SMART_CACHE advances the generation, clears rewrite state, clears the cache, and returns an eight-byte big-endian removed-entry count. Relay bypass or TTL-cap changes clear cache; relay bypass also resets the active resolver. Fingerprint changes isolate entries from earlier resolver selections.

Source: apple/GeauxExtension/PacketTunnelProvider.swift: CacheKey, DNSIngressLimiter, makeCacheKey, resolveAndCache, refreshDNSStateAfterReload, handleAppMessage; apple/GeauxCore/ThreadSafeLRUCache.swift; DNSRequestCoordinator.swift

DoH and DNSCrypt transports

Both resolver implementations return `ValidatedDNSMessage` through the shared `UpstreamResolver` protocol. They differ substantially below that boundary.

DoH pipeline

`DoHResolver` uses a shared ephemeral `URLSession` and `POST` with `Content-Type` and `Accept` `application/dns-message`. Request timeout is three seconds; resource timeout is five. `DoHChunkCollector` requires HTTP 200 and DNS media type, rejects redirects, and caps advertised and accumulated response bytes at 65,535. Chunk collection preserves connection pooling without per-byte async work.

DNSCrypt bridge

`DNSCryptResolver` sends the query, resolver stamp, optional relay stamp, cancellation token, and retained callback context through `geaux_resolve_dnscrypt`. `forceSkipRelay` passes no relay stamp. The Swift timeout is four seconds; completion/cancellation is exactly-once. `ERR:-`-prefixed callback payloads are converted to errors; other callback data goes through `DNSMessageValidator`.

DNSCrypt cryptographic stages

Rust parses the stamp and optional relay destination, retrieves a provider TXT certificate, validates reply identity, checks certificate validity and Ed25519 signature, and selects the supported construction. It derives the shared secret from the ephemeral client key and server key, pads the DNS query, and authenticates/encrypts it. This snapshot accepts certificate construction 0x0002 and implements the DNSCrypt XChaCha20-Poly1305 construction; it does not implement the older 0x0001 construction.

Relay framing and return

An anonymized relay frame prefixes resolver address/port to the encrypted packet. Certificate retrieval also has a relay-aware transport path. Responses are checked for expected format, nonce/authentication, and valid padding before plaintext DNS returns to Swift. The certificate LRU holds at most 1,000 entries keyed by resolver stamp, shared across direct and relay use. A per-resolver refresh lock coalesces cold certificate discovery; expiry is checked before reuse.

Bounds and fallback

The Rust entry point caps queries at 3,967 bytes before padded transport and admits at most 256 in-flight DNSCrypt operations. UDP transport has timeout/cancellation checks; a DNSCrypt TCP fallback path handles framed partial reads within deadlines. This transport fallback does not implement DNS-over-TCP interception from client applications.

Failure surface

Transport, timeout, stamp, certificate, authentication, malformed DNS, and DNS RCODE failures converge on the provider's typed failure reporting. The code does not automatically fail over through all saved resolvers when the active one fails.

Source: `apple/GeauxCore/UpstreamResolver.swift`; `DoHResolver.swift`; `DNSCryptResolver.swift`; `DNSFailureReason.swift`; `geaux-rust/src/lib.rs`: `geaux_resolve_dnscrypt`, `CachedCert`, `CertificateTransport`

WireGuard processing and pacing

Rust owns WireGuard UDP I/O and BoringTun state. Swift primarily feeds captured IP packets and consumes decrypted IP packets.

Outbound pipeline

WireGuardSocketManager.queueEncrypt batches caller-owned bytes into `geaux_wg_queue_encrypt_batch`. Accepted packets acquire byte-budget ownership and enter bounded work queues. The engine enforces outbound destination AllowedIPs, encapsulates with BoringTun, and sends network output through the UDP path. Protocol and endpoint controls remain distinct from application payload processing.

Inbound pipeline

The Rust UDP worker checks plausibility before allocating/queuing accepted input, then BoringTun authenticates and decrypts it. Inbound source AllowedIPs policy is enforced before plaintext reaches the TUN queue. The Swift blocking poller receives plaintext batches, applies the configured response transformer, determines IP family, and writes to packetFlow. `geaux_wg_queue_decrypt` is a legacy no-op; live receive is socket-owned in Rust.

Token bucket

Tokens refill at configured bytes per second, capped by bucket capacity. A packet consumes tokens equal to its byte length. A full bucket permits a burst; once depleted, the worker waits for refill or shutdown. Queue residence is bounded by a 250 ms pacing deadline, after which an undeliverable paced item is dropped. Rate zero bypasses pacing, not all resource limits.

Memory bounds

An aggregate 8 MiB payload budget is accounted by RAII tokens as buffers move through queues. Bounded channel capacity, pacing queue accounting, protocol plausibility checks, and deadline drops are additional protections. These bounds do not mean the entire NetworkExtension process uses only 8 MiB.

Timers and responsiveness

Protocol timers run at 100 ms while active. Idle polling starts at 250 ms and backs off, bounded according to keepalive configuration (up to five seconds without keepalive). Shutdown notification wakes blocking paths. The Swift receive thread blocks for work rather than busy polling.

Metrics

Counters distinguish outbound and receive queue drops, UDP send errors, outbound/inbound policy drops, pacing queue and deadline drops, TUN output drops, missing endpoint, byte-budget rejection, and implausible input. Gauges report current/high-water queued payload and pacing bytes; residence metrics expose queue delay. Packet delivery counters do not certify end-to-end application success.

Source: `geaux-rust/src/wireguard.rs`: WgState, TokenBucket, ByteBudgetToken, `geaux_wg_init`, `geaux_wg_drop_count`; `apple/GeauxCore/WireGuardSocketManager.swift`; `apple/GeauxExtension/PacketTunnelProvider.swift`: `GET_RUNTIME_METRICS`

Persistence, reporting, and lifecycle

Network work, reporting, and UI refresh have separate lifetimes and limits. Traffic history is observational and may be incomplete under pressure.

Traffic write path

NetworkFlow records carry domain, protocol, port, action, matched rule, latency, cache status, qtype, outcome, destination IP, and failure reason. TrafficLogger pushes them into a 4,096-entry ring buffer. A task drains every two seconds into a serialized TrafficBatchWriter; stopAndFlush waits for the worker and drains the final records.

SQLite transactions

TrafficDatabase uses SQLite WAL and prepared statements. Batch inserts and aggregate UPSERTs commit together; errors roll back. Transient contention is retried after 100, 250, and 500 ms (initial attempt plus up to three retries). Permanent or exhausted failures increment dropped-log metrics. UI reads recent rows and aggregates rather than loading packet payloads.

Retention and UI

Maintenance is triggered after 500 inserts or 60 seconds at a write opportunity. It incrementally prunes rows older than a day and overflow beyond the retained-row target of 100,000; bounded deletion means these are maintenance targets, not instant hard caps. TrafficViewModel groups records and preserves type-specific outcomes. A group with success and failure is presented as partial rather than a total DNS failure.

Configuration and secrets

ConfigurationWorker stages/compiles rule data and ConfigurationManager applies changes through the provider. WireGuardSecretStore keeps private and preshared keys in Keychain; normal serialized configuration records references. Copying raw config alone is not a portable WireGuard-secret backup. UI appearance uses separate preferences.

Reload transaction

RELOAD_CONFIG loads a candidate, activates compiled rules, and can restore accepted rules on failure. Mode, profile, or pacing differences require network settings/engine changes; otherwise the fast-path snapshot is published without a WireGuard restart. DNS policy updates advance generation and clear rewrite state.

Network changes and sleep

Effective path identity and recovery generations debounce/cancel obsolete work. Recovery resolves endpoints and can rebuild the UDP engine, then checks an authenticated session. Sleep/wake tracks meaningful path changes; idle age alone is not proof of a broken tunnel. Demand-driven handshake validation can request recovery if traffic cannot establish a session. Actual Wi-Fi/cellular continuity still requires device testing.

Source: apple/GeauxCore/TrafficLogger.swift; RingBuffer.swift; TrafficDatabase.swift; ConfigurationManager.swift; ConfigurationWorker.swift; WireGuardSecretStore.swift; apple/Geaux/TrafficViewModel.swift; apple/GeauxExtension/PacketTunnelProvider.swift: reload/recovery/sleep/wake

Implementation limits and maintenance checklist

Use these distinctions when maintaining the code or publishing feature descriptions. They are grounded in the supplied snapshot, not a security certification.

Topic	Documented boundary
WireGuard-only rewrite	Source-traced unreachable DNS rewrite branch with eval:nil. Fix/validate before promising enforcement.
Resolver egress	No guaranteed nesting of selected encrypted resolver traffic inside WireGuard. DNSCrypt endpoint exclusions are explicit.
DNS endpoint hot reload	Non-WireGuard-runtime reload path publishes DNS settings without reapplying network settings; changed route exclusions may need reconnect.
Local discovery	The setting can return REFUSED on intercepted names, beyond hiding logs.
Automatic MTU	Fixed 1280, not dynamic per-network MTU discovery.
Rules	Hash-based domain matching; unsupported Adblock syntax is skipped. Custom priority applies at identical keys, with specificity still relevant.
DNS transport	Client TCP/fragmented DNS interception unsupported; large responses and TCP-required cases are limited.
Validation	DNS structural and query-identity validation does not verify DNSSEC signatures locally.
Privacy / history	Local histories are bounded; Clear Statistics leaves logs. Neither logs nor counters are exhaustive packet evidence.

Verification performed for this documentation

Source inspection across UI settings, configuration defaults, packet routing, FFI declarations, rule compilation, DNS transports, cache/coalescing, WireGuard workers, and SQLite reporting. Generated documents and website packaging receive layout/link checks. No on-device VPN, Xcode build, resolver reachability, or network-transition tests were performed for this documentation-only task.

Update procedure

When settings change, compare GeauxConfig defaults with UI labels, trace save/reload behavior, update the user and technical chapters, regenerate PDFs and website content, and verify internal links. For ABI changes, compare the C GeauxEvaluation layout and signatures to Rust repr(C) and Swift callers. Re-run repository regression scripts when changing implementation, plus real-device tests for routing modes, IPv4/IPv6, network changes, relays, and large DNS answers.

Source: Source files cited on preceding pages; tests and scripts in the supplied archive. See source-manifest.json and README.txt in the complete documentation package.